

Caffeine의 Window TinyLFU 알고리즘

Caffeine의 Window TinyLFU 알고리즘

Caffeine Cache가 탁월한 성능을 발휘하는 핵심 비밀은 **Window TinyLFU 알고리즘**에 있다. 이 알고리즘은 기존 LRU, LFU의 단점을 모두 해결하면서도 메모리 사용량을 최소화하고, 실무 환경에서 최고의 캐시 적중률을 제공한다. 이 문서에서는 TinyLFU의 동작 원리와 기존 알고리즘 대비 우수성을 상세히 다룬다.

1. TinyLFU란 무엇인가

TinyLFU(Tiny Least Frequently Used)는 최근에 얼마나 자주 조회되었는지를 아주 작은 메모리 사용량으로 지속적으로 추적하는 빈도 기반 캐시 교체 알고리즘이다.

TinyLFU = LFU + Bloom Filter 기반 빈도 추정 + Window/Probation/Protected 영역 결합

핵심 특징

- **정확도**: LFU 수준의 높은 판단 정확도
- **메모리 효율성**: LRU 수준의 낮은 메모리 사용량
- **성능**: 극도로 빠른 $O(1)$ 연산

2. 기존 캐시 알고리즘의 한계

2.1 LRU (Least Recently Used)

동작 방식: 가장 오래 사용되지 않은 항목을 제거한다.

문제점: Cache Pollution(캐시 오염)

핵심 데이터(A, B, C)가 캐시에 있는 상태에서
일회성 데이터(X, Y, Z)가 대량으로 유입되면
→ 핵심 데이터가 밀려나고 일회성 데이터로 캐시가 채워짐
→ 캐시 히트율 급격히 하락

LRU는 최근 접근 시점만 고려하고 접근 빈도는 무시하므로, 일시적으로 대량 유입된 데이터가 자주 사용되는 핵심 데이터를 쉽게 밀어낼 수 있다.

2.2 LFU (Least Frequently Used)

동작 방식: 가장 적게 사용된 데이터를 제거한다.

문제점: Stale Data 문제

과거에 자주 사용되었던 데이터(카운트: 1000)
현재는 거의 사용되지 않음
→ 하지만 높은 카운트 때문에 캐시에서 제거되지 않음
→ 새로운 인기 데이터가 캐시에 진입하지 못함

LFU는 과거 누적 빈도만 보기 때문에, 현재는 사용되지 않지만 과거에 많이 사용된 데이터가 캐시를 점유하는 문제가 있다.

2.3 Segmented LRU (SLRU)

동작 방식: Protected와 Probation 영역으로 나누어 LRU의 문제를 완화한다.

한계점

- 여전히 최근성(Recency)만 판단할 뿐 사용 빈도(Frequency)는 고려하지 못한다.
- 일회성 데이터가 Protected 영역에 진입하면 핵심 데이터를 밀어낼 수 있다.

3. TinyLFU의 해결 방법

TinyLFU는 기존 알고리즘의 모든 문제를 체계적으로 해결한다.

3.1 LRU의 캐시 오염 문제 해결

Window Cache + LFU 조합

- 신규 데이터는 먼저 Window Cache에 진입하여 LRU 방식으로 관리된다.
- Window에서 일정 횟수 이상 사용되면 Main Cache로 승격된다.
- 일회성 데이터는 Window에서 제거되어 Main Cache를 오염시키지 않는다.

3.2 LFU의 Stale Data 문제 해결

주기적인 빈도 카운터 감쇠(Decay)

- 모든 항목의 빈도 카운터를 주기적으로 절반으로 줄인다.
- 과거에 인기 있었지만 현재는 사용되지 않는 데이터의 카운트가 점진적으로 감소한다.
- 최근 패턴이 반영되어 현재 인기 있는 데이터가 우선권을 갖는다.

3.3 메모리 과다 사용 문제 해결

Bloom Filter 기반 Approximate Counter

- 정확한 빈도 카운터 대신 확률적 근사치를 사용한다.
- Count-Min Sketch 자료구조로 구현되어 메모리 사용량이 극소화된다.

- 엔트리당 4~16bit 수준의 메모리만 사용한다.

정확한 숫자를 몰라도 "누가 더 많이 사용되었는지"만 알면 캐시 교체 판단에는 충분하다.

3.4 캐시 적중률 최대화

실험적으로 검증된 바에 따르면 TinyLFU는 다양한 워크로드에서 가장 높은 Hit Rate를 제공한다.

4. TinyLFU의 내부 구조

4.1 Frequency Sketch

TinyLFU는 정확한 빈도 카운터를 저장하지 않는다.

특징

- **Bloom Filter 기반**: Approximate Frequency Counter를 사용한다.
- **메모리 효율**: 아주 작은 메모리만 사용하면서도 교체 판단에 충분한 정확도를 제공한다.
- **Count-Min Sketch**: 다차원 해시 테이블 형태로 빈도를 추정한다.
- **O(1) 연산**: 빈도 조회 및 증가가 상수 시간에 처리된다.

예시:

실제 빈도: A=100, B=95

Sketch 추정치: A=102, B=93

→ "A가 B보다 많이 사용됨"을 정확히 판단 가능

→ 교체 결정에는 충분한 정보

메모리를 절약하면서 속도를 극대화하는 핵심 메커니즘이다.

4.2 Window Cache

- **역할**: 갓 들어온 새로운 데이터를 잠시 저장한다.
- **관리 방식**: LRU 방식으로 관리된다.
- **목적**: 신규 데이터가 캐시에서 완전히 배제되지 않도록 보장한다.

LRU의 장점(신규 데이터 보호)과 LFU의 단점(신규 데이터 진입 어려움)을 동시에 해결한다.

4.3 Main Cache (Probation + Protected)

4.3.1 Probation 영역

- Window에서 승격된 데이터가 처음 진입하는 영역이다.
- 자주 사용되면 Protected 영역으로 승격된다.
- 사용 빈도가 낮으면 제거 후보가 된다.

4.3.2 Protected 영역

- 핵심 데이터가 저장되는 영역이다.
- 접근 빈도가 높은 데이터가 집중적으로 모인다.
- 일정 크기를 초과하면 Probation으로 강등된다.

4.4 제거 결정 프로세스

```
if (캐시가 가득 차서 제거가 필요한 경우) {  
    1. Probation에서 가장 오래된 항목(victim) 선택  
    2. 새로 들어온 후보(candidate)와 빈도 비교  
  
    if (FrequencySketch[candidate] > FrequencySketch[victim]) {  
        // 새 데이터가 더 자주 사용됨  
        victim 제거 후 candidate 삽입  
    } else {  
        // 기존 데이터가 더 가치 있음  
        candidate 거부  
    }  
}
```

빈도가 낮은 데이터만 정확히 제거하여 캐시 효율을 극대화한다.

5. 왜 다른 알고리즘을 사용하지 않는가

5.1 최고의 캐시 적중률 (실험적 검증)

TinyLFU는 다음 알고리즘들과 비교하여 대부분의 워크로드에서 월등한 Hit Rate를 보였다.

- LRU (Least Recently Used)
- LFU (Least Frequently Used)
- ARC (Adaptive Replacement Cache)
- SLRU (Segmented LRU)
- FIFO (First In First Out)
- CLOCK
- 2Q

실제 벤치마크 결과

Caffeine 개발자인 Ben Manes가 실제 트위터, 구글, 아마존의 트래픽 패턴을 사용한 실험에서 TinyLFU 기반 Caffeine이 평균 **20~50% 높은 적중률**을 보였다.

학술 논문과 Google Engineering 벤치마크에서 지속적으로 증명되었다.

5.2 극소량의 메모리 사용

정확한 카운터를 저장하면 엔트리당 수십~수백 바이트가 필요하지만, TinyLFU는 다음을 활용한다.

- **Approximate Frequency Sketch** 사용
- **엔트리당 4~16bit** 수준
- 거의 무시할 정도의 메모리 오버헤드로 빈도 기반 캐싱 구현

5.3 O(1) 고성능

- 모든 연산이 **상수 시간(O(1))**에 처리된다.
- Lock 경합이 거의 없다.
- 특히 Caffeine은 멀티 코어 환경에서 비교 불가능한 수준의 성능을 제공한다.

5.4 신규/기존 데이터 균형

기존 알고리즘들은 신규 데이터 또는 기존 데이터 중 하나만 우대하는 편향이 있었으나, TinyLFU는 두 문제를 모두 해결하였다.

문제	LFU	LRU	TinyLFU
신규 데이터 보호	✗	✓	✓
오래된 인기 데이터 보호	✓	✗	✓
일회성 데이터 유입 차단	✗	✗	✓

5.5 실무 환경 최적화

현실적인 워크로드 특성을 반영한다.

- **Zipfian 분포**: 일부 데이터가 대부분의 요청을 차지하는 패턴
- **시간에 따른 인기도 변화**: 과거 인기 데이터가 현재는 사용되지 않는 패턴
- **버스트 트래픽**: 일시적으로 특정 데이터에 대량 요청이 몰리는 패턴

이 모든 시나리오에서 TinyLFU가 가장 뛰어난 성능을 보인다.

6. TinyLFU가 사실상 표준이 된 이유

종합 평가

- ✓ 메모리를 적게 사용한다
- ✓ 빈도 계산이 정확하다
- ✓ LRU의 약점(캐시 오염)을 해결하였다
- ✓ LFU의 약점(stale data)을 해결하였다

- ✓ 최신 워크로드에서 최고의 캐시 적중률을 제공한다
 - ✓ $O(1)$ 성능으로 극도로 빠르다
 - ✓ JVM 환경에서 동시성 문제를 최소화하였다
-